

Neural PDE Solvers for the 1D Elastic Wave Equation

Technical Reference: Problem Setup, PINN Arm, and the Fourier Neural Operator
Baselines

SciML Project — Ayush Debnath

July 2026

Contents

1	The Physical Problem	2
2	Nondimensionalization	2
3	Material Models	2
4	Initial Condition and the Hard-Constraint Ansatz	3
5	The Finite-Difference Reference Solver	4
6	The PINN Arm in Brief	4
7	Why Operator Learning	5
8	FNO Dataset Generation (wave/run_fno_baseline.py)	5
9	Input Encoding	7
10	The Spectral Layer (SpectralConv2d)	7
11	Architecture (FNO2d) and Parameter Accounting	8
12	Training Protocol	10
13	The Control Experiment (ConvBaseline2d)	10
14	Evaluation Protocol and Results	10
15	The Second FNO: Ansatz-Conditioned (final/ML)	11
16	Quick-Reference Numbers	13

1 The Physical Problem

The project solves the one-dimensional elastic wave equation in *conservative form*:

$$\rho(x) u_{tt} = \partial_x [E(x) u_x], \quad x \in [x_{\min}, x_{\max}], \quad t \in [0, 1], \quad (1)$$

where $u(x, t)$ is displacement, $E(x)$ the stiffness profile, and $\rho(x)$ the density. The local wave speed is $c(x) = \sqrt{E(x)/\rho(x)}$.

The conservative form matters. Expanding the flux term gives $E(x) u_{xx} + E'(x) u_x$; the naive reduction $u_{tt} = c^2 u_{xx}$ silently drops the $E'(x) u_x$ term and therefore gets reflection and transmission at material interfaces wrong. Every solver in the repository (finite differences, PINN losses, FNO training data) uses the conservative form.

Intuition: a wave hitting a stiffness jump partially bounces back and partially passes through. The dropped term is exactly the part of the physics that encodes this splitting.

2 Nondimensionalization

All computation happens in nondimensional units, defined per material (`wave/materials.py`, class `MaterialModel`). Given the physical domain $[x_{\min}^{\text{phys}}, x_{\max}^{\text{phys}}]$ and reference constants $E_{\text{ref}}, \rho_{\text{ref}}$:

$$L_{\text{ref}} = \frac{1}{2} (x_{\max}^{\text{phys}} - x_{\min}^{\text{phys}}), \quad V_{\text{ref}} = \sqrt{E_{\text{ref}}/\rho_{\text{ref}}}, \quad T_{\text{ref}} = L_{\text{ref}}/V_{\text{ref}}, \quad (2)$$

so that $x \in [-1, 1]$, wave speeds are $O(1)$, and all PDE coefficients are order one. Conversions back to physical units are $x^{\text{phys}} = x L_{\text{ref}}$, $t^{\text{phys}} = t T_{\text{ref}}$, and the source width scales as $\sigma_g = \sigma_g^{\text{phys}}/L_{\text{ref}}$.

Intuition: neural networks train badly when inputs and coefficients span wildly different scales (stiffness in the tens vs. coordinates in $[-1, 1]$). Nondimensionalizing is one of the standard “make PINNs actually work” steps.

3 Material Models

Three canonical materials of increasing difficulty (`wave/materials.py`); density is $\rho \equiv 1$ in all of them, so only stiffness varies and $c(x) = \sqrt{E(x)}$.

Model	Physical domain	E_{ref}	Stiffness profile (nondim.)
Homogeneous	$[-1, 1]$	80	$E = 1$ everywhere
TwoLayer	$[-1, 1]$	80	$1.0 \rightarrow 1.5$ jump at $x = 0$
MultiLayer	$[-1.5, 1.5]$	60	6 layers, $1.0 \rightarrow 2.5$ staircase

Smoothed interfaces. Material jumps are represented by a differentiable tanh switch rather than a true discontinuity. For a single interface at x_b with transition width $w = 0.02/L_{\text{ref}}$:

$$\alpha(x) = \frac{1}{2} \left(1 + \tanh \frac{x - x_b}{w} \right), \quad E(x) = E_L (1 - \alpha) + E_R \alpha. \quad (3)$$

α runs smoothly from 0 (left material) to 1 (right material) over a width of a few w . The `MultiLayer` model chains this blend across its five interfaces (at $x = -\frac{2}{3}, -\frac{1}{3}, 0, \frac{1}{3}, \frac{2}{3}$ in nondimensional units, values $E \in \{1.0, 1.3, 1.6, 1.9, 2.2, 2.5\}$). Because interfaces are ~ 0.33 apart while transitions are ~ 0.013 wide, each blend reduces locally to a clean two-value crossfade.

Intuition: the PINN loss needs $\partial_x [E u_x]$, so $E(x)$ must be differentiable — a step function has no usable gradient. The FD solver instead handles jumps with harmonic averaging (Section 5); two different answers to the same problem.

The three canonical materials (tanh-smoothed interfaces, $w \approx 0.013$)

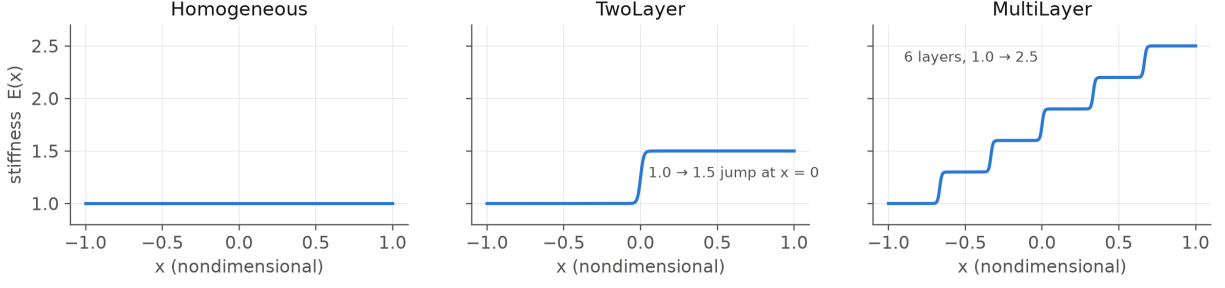


Figure 1: Stiffness profiles $E(x)$ of the three canonical materials. All interfaces are tanh-smoothed with width $w \approx 0.013$.

4 Initial Condition and the Hard-Constraint Ansatz

Initial displacement. A normalized first derivative of a Gaussian (`gaussian.ic` in `wave/problem_data.py`), an antisymmetric single-wiggle pulse of width $\sigma_g = 0.1$ centred at $x_0 = 0$:

$$f(x) = \exp\left(-\frac{(x-x_0)^2}{2\sigma_g^2}\right), \quad g(x) = \frac{f'(x)}{\max_x |f'(x)|}, \quad u(x, 0) = g(x), \quad u_t(x, 0) = 0. \quad (4)$$

The normalization makes the peak amplitude exactly 1 (raw $|f'|$ peaks at $(1/\sigma_g) e^{-1/2} \approx 6.07$ at $x = x_0 \pm \sigma_g$).

The ansatz. Instead of penalizing initial-condition errors, the solution is *assumed to have the form*

$$\hat{u}(x, t) = \underbrace{g(x) e^{-\frac{1}{2}(15t)^2}}_{\text{known IC, fading out}} + \underbrace{\tanh^2(25t)}_{\text{gate}} \cdot \underbrace{N_\theta(x, t)}_{\text{learned "NN term"}}. \quad (5)$$

Both initial conditions hold *exactly, by construction*:

- At $t = 0$: the exponential equals 1 and $\tanh^2(0) = 0$, so $\hat{u}(x, 0) = g(x)$ regardless of the network output.
- For velocity: $\frac{d}{dt} e^{-\frac{1}{2}(15t)^2} = -225 t e^{(\cdot)} \rightarrow 0$ and $\frac{d}{dt} \tanh^2(25t) = 50 \tanh(25t) \text{sech}^2(25t) \rightarrow 0$ at $t = 0$ (the *squaring* of the tanh is what kills the slope), so $\hat{u}_t(x, 0) = 0$ exactly.

The constants 15 and 25 set the handoff window: the IC term is negligible by $t \approx 0.2$ and the gate is fully open by $t \approx 0.1$. The network therefore owns $\sim 90\%$ of the time domain, and the optimizer never has to fight for the initial condition. All derivatives in the PDE residual are taken through the *assembled* $\hat{u}$, not the raw network.

$$\hat{u}(x, t) = \underbrace{g(x) e^{-\frac{1}{2}(15t)^2}}_{\substack{\text{known IC, fading out} \\ = g(x) \text{ at } t=0; \\ \text{gone by } t \approx 0.2}} + \underbrace{\tanh^2(25t)}_{\substack{\text{gate} \\ 0 \text{ at } t=0 \text{ with zero slope} \\ \text{(the square kills the} \\ \text{slope); open by } t \approx 0.1}} \cdot \underbrace{N_\theta(x, t)}_{\substack{\text{learned "NN term"} \\ \text{the blank in the template} \\ \text{(PINNs and final/ML} \\ \text{FNO both fill it)}}$$

Both initial conditions hold exactly, by construction: $\hat{u}(x, 0) = g(x)$ and $\hat{u}_t(x, 0) = 0$

Figure 2: Anatomy of the hard-constraint ansatz: the template owns $t = 0$, the network owns everything after.

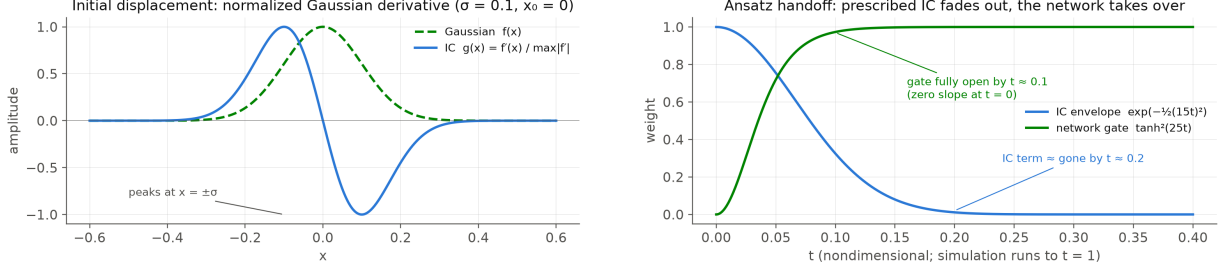


Figure 3: Left: the initial pulse $g(x)$ (normalized Gaussian derivative) and its parent Gaussian. Right: the two time gates of the ansatz — the prescribed IC fades out while the network gate opens.

Intuition: an ansatz is a fill-in-the-blanks template: “the solution starts as this exact pulse, fades that in and out smoothly, and here is a blank for whatever happens afterwards.” The network learns only the blank — the NN term. Its raw output looks nothing like the wave; it is the correction the template still needs.

5 The Finite-Difference Reference Solver

`wave/fd_solver.py` provides ground truth for everything: PINN scoring, FNO training labels, and FNO evaluation. It integrates (1) with an explicit second-order leapfrog scheme in flux form. With $\lambda_i = \Delta t^2 / (\rho_i \Delta x^2)$:

$$u_i^{n+1} = 2u_i^n - u_i^{n-1} + \lambda_i \left[E_{i+\frac{1}{2}} (u_{i+1}^n - u_i^n) - E_{i-\frac{1}{2}} (u_i^n - u_{i-1}^n) \right], \quad (6)$$

with *harmonic-mean* interface stiffness

$$E_{i+\frac{1}{2}} = \frac{2 E_i E_{i+1}}{E_i + E_{i+1}}, \quad (7)$$

which preserves correct transmission/reflection behaviour across interfaces. The first step uses a Taylor start $u_i^1 = u_i^0 + \Delta t g_i + \frac{1}{2} \lambda_i [\text{flux}]_i^0$.

Stability. The time step obeys the CFL condition $\Delta t = \text{CFL} \cdot \Delta x / \max_x c(x)$ with safety factor $\text{CFL} = 0.9$; the solver picks its own Δt and callers re-interpolate onto their target time grid.

Absorbing boundaries. First-order radiation conditions $u_t \mp c u_x = 0$ discretized with the *Mur* scheme,

$$u_0^{n+1} = u_1^n + \beta (u_1^{n+1} - u_0^n), \quad \beta = \frac{c \Delta t - \Delta x}{c \Delta t + \Delta x}, \quad (8)$$

mirrored on the right boundary. (The seemingly natural centred-time / one-sided-space discretization is unconditionally unstable.)

Why FD as the teacher. No closed-form solution exists for layered media; FD is provably convergent with understood error and stability; it is the *single* referee for all arms of the project, keeping rankings meaningful; and in 1D it is cheap enough to run hundreds of times. (In 3D the economics flip — which is precisely when operator learning starts to pay.)

6 The PINN Arm in Brief

Seven architectures share the identical ansatz (5), losses, and two-phase training (Adam $\rightarrow$ L-BFGS):

Architecture	Key idea
VanillaPINN	MLP, tanh activations, Xavier init
FourierFeaturePINN	trainable random Fourier embedding $\rightarrow$ MLP
PirateNet	fixed Fourier embed, RWF layers, residual blocks ($\alpha=0$ init)
WavKAN	wavelet-based Kolmogorov–Arnold network
KAN (pykan)	spline-based KAN
ChebyshevKAN	Chebyshev-recurrence edges (cPIKAN)
FourierWavKAN	Fourier embedding $\rightarrow$ WavKAN trunk

Training machinery (`src/losses/wave.loss.py`), summarized:

- **Causal weighting** (Wang et al. 2022): time split into 16 slabs; slab m weighted by $w_m = \exp(-\varepsilon \sum_{k < m} L_k)$ so late times only count once early times are solved.
- **GradNorm** (Wang et al. 2023): EMA-smoothed gradient-norm balancing of $w_{\text{pde}}, w_{\text{bc}}, w_{\text{ic}}$, updated every 100 steps.
- **RBA**: pointwise residual-based attention $\lambda \leftarrow 0.999\lambda + 0.01 |r| / \max |r|$, $\text{loss} = \overline{(\lambda r)^2}$.
- **SOAP**: Adam in the Shampoo eigenbasis (2D parameters only).
- **Absorbing BCs** as loss terms; checkpoint selection by a stationary validation metric; L-BFGS runs with causal weighting off.

The essential property for what follows: *a PINN is trained per material*, taking hours per case, with elaborate machinery required to make the physics loss optimizable at all.

7 Why Operator Learning

A PINN answers one question: “what is $u(x, t)$ for *this* material and *this* pulse?” The Fourier Neural Operator (FNO; Li et al. 2020) instead learns the *map*

$$\mathcal{G} : (E(x), \rho(x), g(x)) \mapsto u(x, t), \quad (9)$$

so a *new* material costs one forward pass ($\sim$ milliseconds) instead of a fresh optimization ($\sim$ hours). The trade: the FNO needs a solver-generated dataset and inherits its training distribution and grid.

	Coordinate solver (PINN)	Neural operator (FNO)
Learns	$(x, t) \mapsto u$, one case	material $\mapsto$ wavefield, a family
Supervision	PDE + BC + IC losses	FD solutions (plain regression)
New material	retrain from scratch	one forward pass
Data required	none	768 FD solves
Mesh	mesh-free	grid-to-grid
Training difficulty	high (causal, GradNorm, ...)	low (MSE + AdamW)

8 FNO Dataset Generation (`wave/run_fno_baseline.py`)

There is no external dataset: the project manufactures its own, using the FD solver as the answer key. The full preset repeats the following recipe 768 times (Figure 5).

1. Sample a random material (`sample_material_profile`). One of four families, uniformly at random; $\rho \equiv 1$ in all:

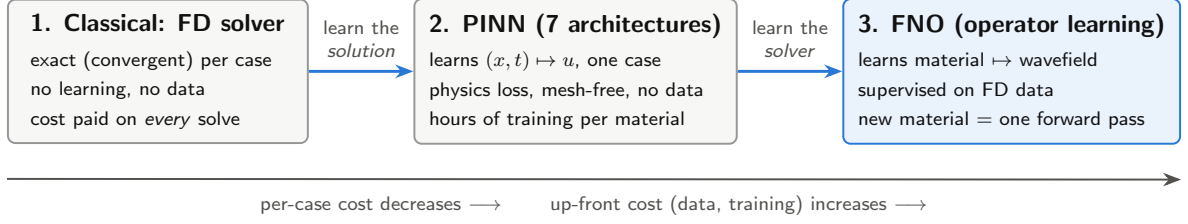


Figure 4: The three solver paradigms in the project, ordered by what they learn: nothing (FD), the solution of one case (PINN), the solver itself (FNO).

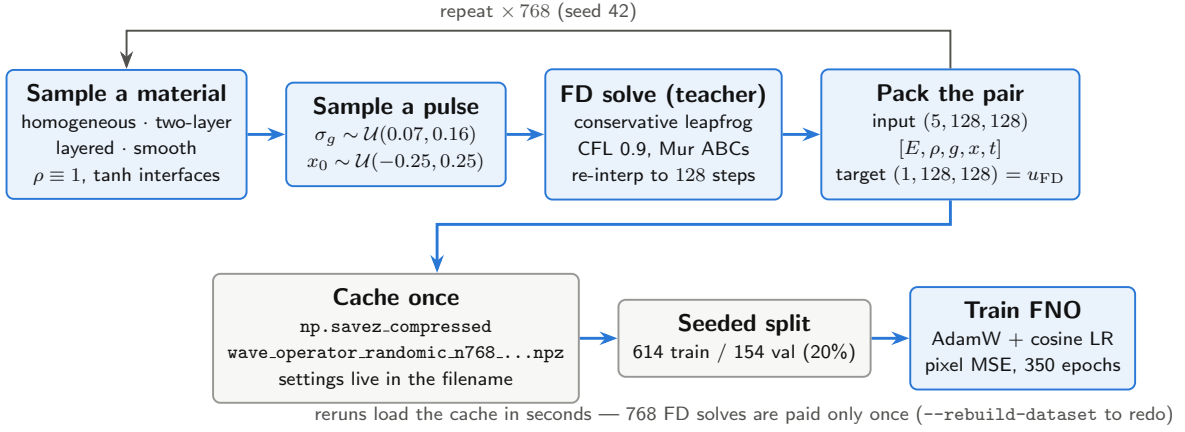


Figure 5: The data factory: sample a material and a pulse, let the FD solver produce the truth, pack the pair, cache everything once.

Family	Sampling
homogeneous	constant $E \sim \mathcal{U}(0.7, 2.2)$
two-layer	$E_L \sim \mathcal{U}(0.7, 1.5)$, $E_R \sim \mathcal{U}(1.0, 2.5)$, boundary $\sim \mathcal{U}(-0.45, 0.45)$, tanh width $\sim \mathcal{U}(0.015, 0.06)$
layered	3–7 layers, values $\sim \mathcal{U}(0.7, 2.5)$; evenly spaced boundaries jittered by $\mathcal{N}(0, 0.04)$; chained tanh blends
smooth	constant $\mathcal{U}(0.9, 1.4)$ plus 3 sine modes with amplitude $\mathcal{U}(-0.18, 0.18)$ and random phase; clipped to $[0.55, 2.5]$

This table *defines the training distribution* — the universe of materials the FNO ever sees (Figure 6). The three canonical materials lie inside these families by design.

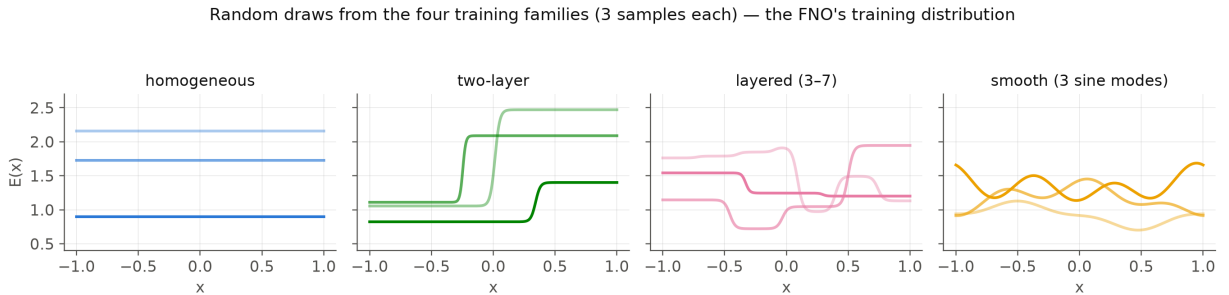


Figure 6: Random draws from the four material families, exactly as `sample_material_profile` produces them.

2. Sample a random pulse (`--random-ic`): $\sigma_g \sim \mathcal{U}(0.07, 0.16)$, $x_0 \sim \mathcal{U}(-0.25, 0.25)$. Prevents memorization of a single source.

3. Solve with FD (`solve_case`). The sampled E, ρ arrays are wrapped as interpolating functions, handed to `solve_wave_1d` (CFL 0.9), and the result is linearly re-interpolated onto the uniform 128-step time grid (the solver chooses its own Δt). Solved in float64, stored as float32.

4. Pack question and answer (Section 9), giving per-sample tensors of shape (5, 128, 128) and (1, 128, 128).

5. Cache. All samples are stacked and written once with `np.savez_compressed` to a self-describing filename,

`wave_operator_randomic_n768_nx128_nt128_t1_seed42.npz`

Every setting that changes the data (IC mode, sample count, grids, horizon, seed) is part of the name, so a stale cache can never be loaded by mistake; `load_or_generate_dataset` loads it in seconds on reruns (or rebuilds with `--rebuild-dataset`). A seeded shuffle then splits 614 train / 154 validation (20%).

9 Input Encoding

`make_input_tensor` lays each case out as a 5-channel image over the (x, t) grid — everything the operator is allowed to know:

$$a(x, t) = [E(x), \rho(x), g(x), x, t] \mapsto u(x, t). \quad (10)$$

- E, ρ, g depend on x only; each column is broadcast (`np.repeat`) across all 128 time steps, so the material is visible at every time slice.
- The x and t *coordinate channels* store each pixel’s own position — the standard FNO trick for positional awareness.
- Layout is channel-first, (batch, channel, x, t) — the PyTorch convention expected by `Conv2d` and `rfft2`.

Batched dataset shapes: inputs (768, 5, 128, 128), outputs (768, 1, 128, 128). Note the FNO predicts the *entire* space–time field in one shot; it is not autoregressive and receives no wavefield frames as input.

10 The Spectral Layer (SpectralConv2d)

Fourier background. `rfft2/irfft2` translate between “value at each grid point” and “how much of each spatial–temporal frequency the field contains.” Each frequency coefficient is a *complex* number — amplitude and phase in one value. For *real* input the negative-frequency half of the last axis is the complex conjugate mirror of the positive half, so `rfft2` stores only $n_t/2 + 1 = 65$ entries along t (nothing is lost; `irfft2` reconstructs the mirror). Along x the full spectrum is kept, with low positive frequencies at the start of the array and low negative frequencies at the end.

The layer. One forward pass of `SpectralConv2d` (64 channels in/out, $m_x = m_t = 20$ modes):

1. $\hat{v} = \text{rfft2}(v)$ — to frequency space (`norm="ortho"`).
2. Allocate the output spectrum as *zeros*: every mode above the cutoff is silently discarded (a learned low-pass filter).

3. **Mix**: for each retained mode k , multiply the 64-vector of channel coefficients by that mode’s own learned complex matrix $W(k) \in \mathbb{C}^{64 \times 64}$:

$$\hat{v}_o^{\text{out}}(k) = \sum_{i=1}^{64} W_{io}(k) \hat{v}_i^{\text{in}}(k), \quad (11)$$

implemented as `einsum("bixy,ioxy->boxy")`. Two separate weight tensors, `weights_pos` and `weights_neg`, handle the $+k$ and $-k$ x -frequency families (slices `[:mx]` and `[-mx:]`) — i.e. right- and left-travelling wave patterns get independent treatment.

4. $v^{\text{out}} = \text{irfft2}(\hat{v}^{\text{out}}, s=(n_x, n_t))$ — back to the grid at the exact input shape.

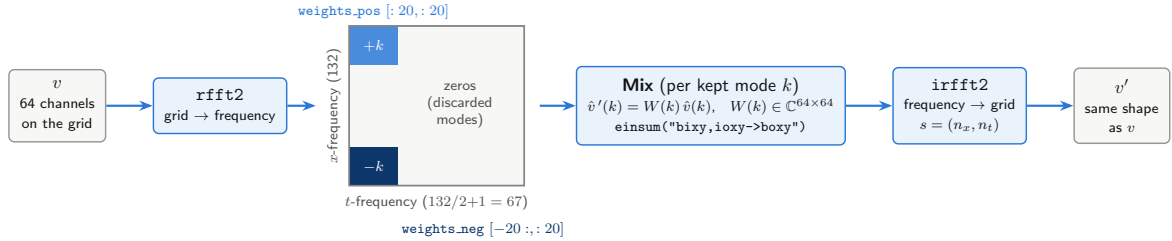
Why complex weights. Multiplying a frequency coefficient by a complex number rescales the wiggle’s amplitude *and* slides its phase — and phase shift is displacement for a wave. “Take the pulse pattern from channel 12, move it right, weaken it 30%, deposit it into channel 5” is one complex multiplication.

Why it is the right tool. By the convolution theorem, pointwise multiplication in frequency space equals a *global* convolution in physical space:

$$(\mathcal{K}v)(x) = \mathcal{F}^{-1} [R_\theta \cdot \mathcal{F}v](x). \quad (12)$$

One layer connects every pixel to every other — matching the physics, where the pulse traverses the entire domain within $t \in [0, 1]$. A 3×3 convolution stack would need ~ 60 layers for the same reach.

What Mix cannot do. Frequencies never talk to each other inside a spectral layer (each mode is transformed independently) — the operation is linear. Cross-frequency interactions, which sharp interfaces require, are created *between* layers by the GELU nonlinearity in grid space. That is the real reason the network stacks four blocks.



Convolution theorem: pointwise multiplication in frequency space = one *global* convolution in physical space

$2 \times 20 \times 20 = 800$ mode slots, each with its own complex 64×64 mixing table $\Rightarrow 13,107,200$ of the 13,132,673 parameters

Figure 7: Inside SpectralConv2d: transform, truncate to the low-mode corners ($+k$ and $-k$ blocks carry `weights_pos/weights_neg`), mix per mode, transform back.

11 Architecture (FN02d) and Parameter Accounting

One Fourier block combines the spectral path with a local per-pixel bypass and the nonlinearity:

$$v \mapsto \text{GELU}(\text{SpectralConv2d}(v) + W_{1 \times 1} v). \quad (13)$$

The 1×1 convolution bypass (a per-pixel linear map) keeps high-frequency content alive past the mode-20 cutoff and acts as a residual-style shortcut. The GELU is the *only* nonlinearity; without it, four stacked blocks would collapse into one linear operation (Figure 8).

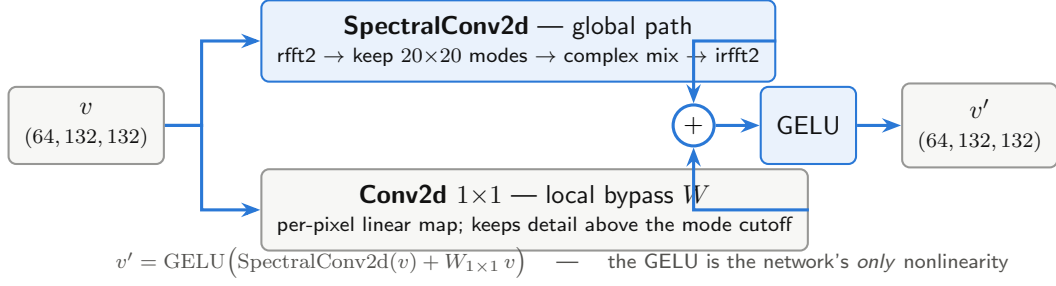


Figure 8: One Fourier block: the global spectral path and the local 1×1 bypass meet in a sum, followed by the GELU.

Full forward pass (full preset: width 64, modes 20×20 , 4 blocks, padding 4):

5-ch input $\rightarrow$ lift 1×1 ($5 \rightarrow 64$) $\rightarrow$ pad $\rightarrow$ block 1 $\rightarrow$ block 2 $\rightarrow$ block 3 $\rightarrow$ block 4 $\rightarrow$ crop $\rightarrow$ project ($64 \rightarrow 128 \rightarrow 1$) $\rightarrow u(x, t)$

The four blocks are structurally identical but share no weights — four independent editing stations, each with its own 800 mixing tables. *Padding*: the FFT implicitly assumes the field is periodic (wraps around like a donut); the wavefield is not, so 4 scrap pixels are appended on the right/bottom before the blocks and cropped after, giving wrap-around artifacts somewhere disposable to land. Figure 9 shows the full stack with tensor shapes and parameter counts.

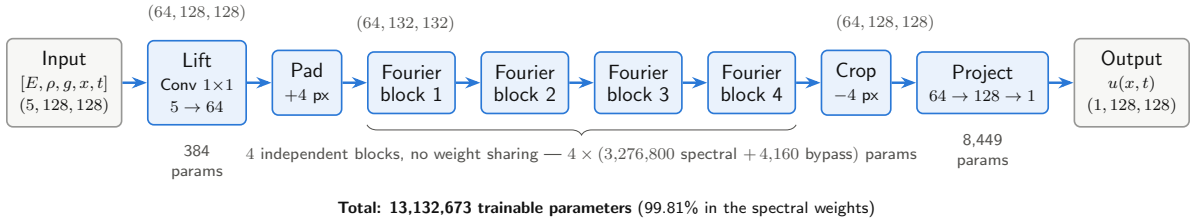


Figure 9: FN02d end to end (full preset): pointwise encoder, four Fourier blocks, pointwise decoder.

Parameter budget (exact). A parameter is one trainable number.

Component	Formula	Count
Spectral weights	$4 \text{ blocks} \times 2 (\pm k) \times (64 \cdot 64 \cdot 20 \cdot 20)$	13,107,200
Local 1×1 bypasses	$4 \times (64 \cdot 64 + 64)$	16,640
Projection head	$(64 \cdot 128 + 128) + (128 \cdot 1 + 1)$	8,449
Lift layer	$5 \cdot 64 + 64$	384
Total	matches <code>run_summary.json</code>	13,132,673

Reading the spectral formula inside-out: one 64×64 mixing table per retained mode (4,096 knobs); $20 \times 20 = 400$ modes each with its own table; $\times 2$ for the positive/negative x -frequency families; $\times 4$ independent blocks. The spectral tables hold 99.81% of all parameters — *the model essentially is its frequency tables*; lift, bypasses and projection are pointwise plumbing.

Counting caveat: `numel` counts each complex weight once; in raw floats the spectral part is ≈ 26 M numbers (≈ 52 MB as float32 pairs). Note also what is absent from the formula: the grid size. Parameters attach to frequencies, not pixels — the architectural fact behind zero-shot evaluation at a different resolution.

12 Training Protocol

Training (`train_model`) is deliberately plain supervised regression:

`zero_grad` $\rightarrow$ `pred = model(x)` $\rightarrow$ `loss = MSE(pred, uFD)` $\rightarrow$ `backward` $\rightarrow$ `step`

- **Loss:** pixelwise mean squared error over all 128×128 points. No physics terms anywhere.
- **Optimizer:** AdamW, learning rate $8 \times 10^{-4} \rightarrow 10^{-5}$ on a cosine schedule over 350 epochs; weight decay 10^{-4} ; batch size 8.
- **Checkpointing:** `model_best.pt` is overwritten only when validation MSE improves and is *reloaded at the end* — the delivered model is the best generalizer, not the last epoch. `model_latest.pt` (with optimizer and scheduler state) is written every 10 epochs so `--resume` survives Kaggle session kills.
- **Mixed precision:** off by default (cuFFT under autocast can be unstable); `torch.compile` optional.

What is absent — on purpose. No PDE residual, no collocation points, no causal weighting, no GradNorm, no RBA, no L-BFGS phase, no ansatz. The PINN machinery exists because the physics loss is hard to optimize; supervision with FD labels removes the difficulty — at the price of a dataset (768 FD solves paid up front).

Kaggle presets (`kaggle/run_fno_t4.py`), one T4 GPU:

Preset	Samples	Train grid	Eval grid	Epochs	Width	Modes	Params
smoke	32	64^2	96^2	10	16	8×8	$\approx 0.1\text{M}$
medium	256	128^2	192^2	150	48	16×16	$\approx 4.7\text{M}$
full	768	128^2	256^2	350	64	20×20	13.1M

13 The Control Experiment (ConvBaseline2d)

A plain CNN with the identical 5-in/1-out tensor interface: six 3×3 convolutions with GELU (`--compare-cnn` trains both models). Each 3×3 layer grows the receptive field by 2 pixels, so after 6 layers each output pixel sees only ≈ 13 pixels — while the wave physically traverses all 128. The CNN *cannot* see the far side of the domain. If the FNO wins, the credit demonstrably belongs to global spectral mixing, not to generic deep-learning capacity.

14 Evaluation Protocol and Results

Metric. Per-sample relative L_2 error, in percent:

$$\varepsilon = 100 \cdot \frac{\|u_{\text{pred}} - u_{\text{FD}}\|_2}{\|u_{\text{FD}}\|_2}. \quad (14)$$

Two exams.

1. **Validation** — 154 held-out random cases at the training resolution (128^2): in-distribution generalization.
2. **Canonical materials** (`evaluate_reference_materials`) — the three official materials with the fixed pulse ($\sigma_g=0.1$, $x_0=0$), evaluated at 256^2 : *twice* the training resolution, zero-shot (parameters live on frequencies, so the same weights run on any grid). FD truth is regenerated on the finer grid.

Results (full preset, fno_t4_full/run_summary.json).

Benchmark	Rel. L_2
Validation (128^2 , in-distribution)	3.21%
Homogeneous (256^2)	28.35%
TwoLayer (256^2)	31.18%
MultiLayer (256^2)	36.44%

Across presets, validation error falls $94.8\% \rightarrow 7.4\% \rightarrow 3.2\%$ (smoke $\rightarrow$ medium $\rightarrow$ full); train MSE reaches 1.0×10^{-6} with validation MSE plateauing at 1.3×10^{-4} — converged, with a widening train/val gap (Figure 10).

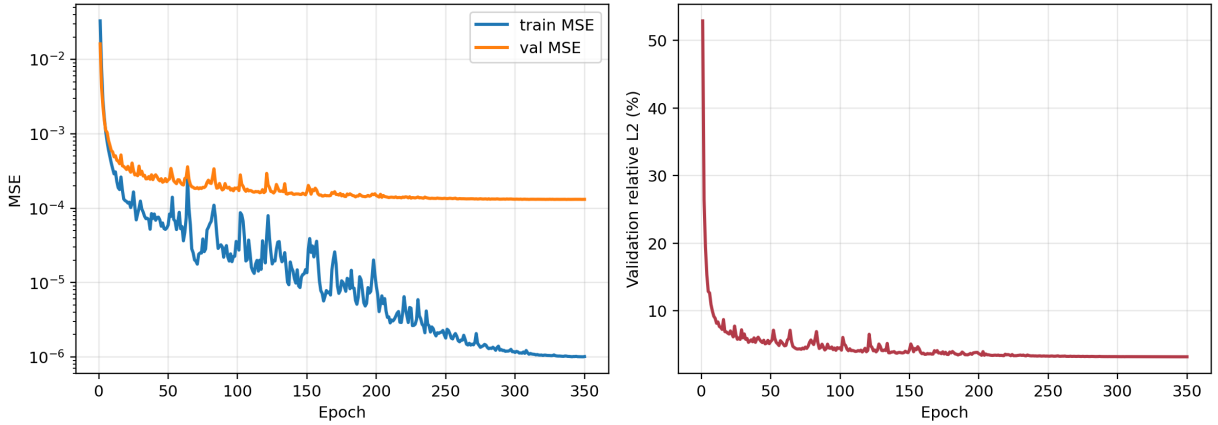


Figure 10: Training history of the full T4 run. Left: train/val MSE (log scale) — the widening gap shows mild overfitting after convergence. Right: validation relative L_2 settling at 3.2%.

The open question. The $3\% \rightarrow 30\%$ gap conflates two changes made simultaneously: (i) zero-shot super-resolution ($128 \rightarrow 256$; the FD reference itself also sharpens at higher resolution), and (ii) the switch from random draws to the fixed canonical cases. The clean diagnostic — re-running the canonical benchmark at the training resolution (`--eval-nx 128 --eval-nt 128`) — decomposes the error into resolution-transfer vs. everything else, and is the first item on the next-runs list. Until then, calling the 36% “out-of-distribution” is imprecise: the canonical materials arguably lie *inside* the sampled families; what mainly changed is the grid.

15 The Second FNO: Ansatz-Conditioned (final/ML)

The repository contains a second, deliberately different FNO (final/ML/src/models.py: FN01d, FNOWrapper; trainer training_code_fno.py):

- **1D, per time slice.** A 1D spectral operator over x only (32 modes, width 64); the query time t enters as an input channel: $[x, E(x), g(x), t] \mapsto$ one spatial slice.
- **Predicts the ansatz NN term, not u .** The raw output is plugged into the blank of the hard-constraint template (5) (`pred = apply_ansatz(nn_out, ...)`) *before* the MSE against FD — so the initial condition is exact by construction and the model inherits the PINN arm’s guarantee.
- **Per-material operator over the pulse width.** Trained on a bank of 32 FD solves of *one* material with $\sigma_g \sim \mathcal{U}(0.05, 0.2)$ (the canonical $\sigma_g = 0.1$ guaranteed in the bank); an operator over initial conditions, not materials.
- **Purpose:** it speaks the pointwise `model(x, t)` interface of `benchmark_v2.py`, so the FNO can sit in the same comparison table as the seven PINN architectures, apples-to-apples.

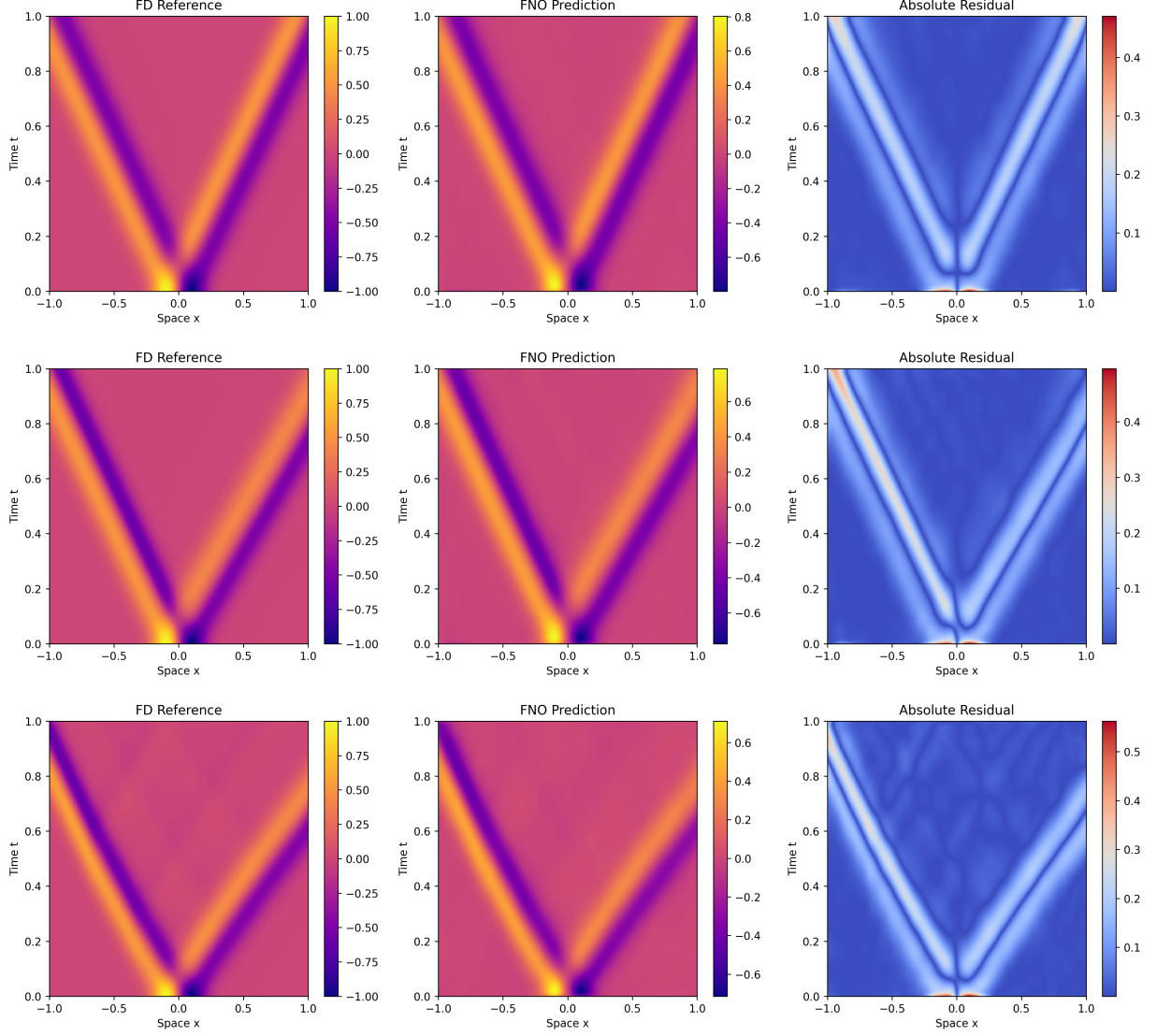


Figure 11: The canonical-material exam at 256×256 (top to bottom: Homogeneous, TwoLayer, MultiLayer). Each row: FD reference | FNO prediction | absolute residual. The wave geometry and amplitudes are tracked well; the residual concentrates on the wavefronts and the $t = 0$ cusp — a phase/sharpness offset rather than a geometry error — and grows with material complexity (28.4%, 31.2%, 36.4%).

Comparison of the two baselines.

	wave/ (grid-to-grid)	final/ML (ansatz-conditioned)
Mapping	$[E, \rho, g, x, t] \rightarrow u(x, t)$, whole field at once	$[x, E, g, t] \rightarrow$ ansatz NN term, slice by slice
Generalizes over	materials and pulses	pulse width, one material
IC handling	learned from data	exact via ansatz
Reported accuracy	3.2% val; 28–36% canonical @ 256^2	2.53% best mean L_2 , exp1 (Homogeneous)
Status	smoke/medium/full converged	exp1 only; exp2/3 not yet run

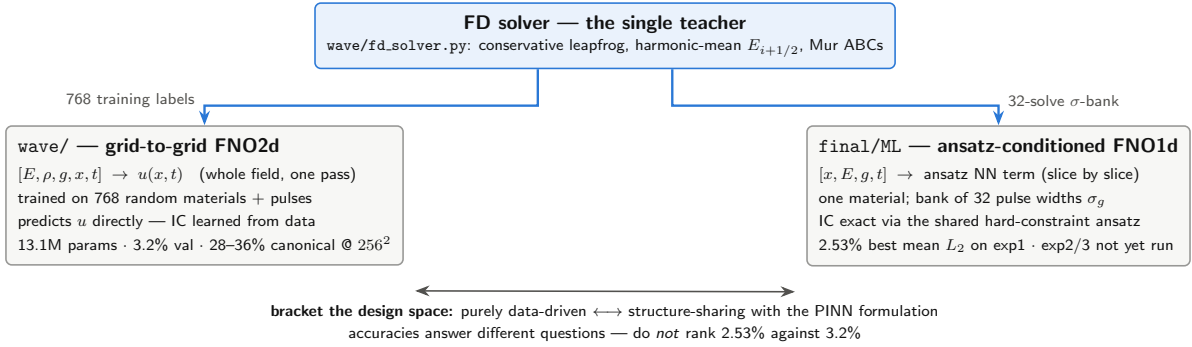


Figure 12: One teacher, two complementary FNO baselines.

Comparability caveat. The 2.53% and 3.2% numbers answer different questions and must not be ranked against each other: exp1 is the easiest material, evaluated at the canonical pulse that was deliberately included in its training bank, while the **wave/** numbers span random materials and a doubled evaluation grid. Likewise, “physics-constrained” overstates the second model: the ansatz enforces only the initial condition exactly; there is still no PDE term in either loss. The honest framing: the two FNOs bracket the design space from purely data-driven (grid-to-grid) to structure-sharing with the PINN formulation (IC-hard-constrained).

16 Quick-Reference Numbers

768 / 614 / 154	samples total / train / validation (full preset)
(768, 5, 128, 128)	input tensor: samples $\times$ channels $\times x \times t$
5 channels	$E(x), \rho(x), g(x), x$ -mesh, t -mesh
$20 \times 20 (\times 2)$	retained modes per axis (positive and negative x -families)
13,132,673	trainable parameters; 99.81% spectral; ≈ 26 M raw floats
$8 \times 10^{-4} \rightarrow 10^{-5}$	AdamW cosine learning-rate schedule, 350 epochs, batch 8
3.21%	validation relative L_2 (in-distribution, 128^2)
28.35 / 31.18 / 36.44%	Homogeneous / TwoLayer / MultiLayer at 256^2 , zero-shot
2.53%	final/ML ansatz-conditioned FNO, exp1 canonical window
CFL = 0.9	FD stability safety factor; harmonic-mean $E_{i+1/2}$; Mur ABCs

Key References

- Li, Kovachki, Azizzadenesheli et al., *Fourier Neural Operator for Parametric Partial Differential Equations*, arXiv:2010.08895 (2020).
- Wang, Sankaran, Perdikaris, *Respecting causality is all you need for training physics-informed neural networks*, arXiv:2203.07404 (2022).
- Wang et al., *An Expert's Guide to Training Physics-Informed Neural Networks*, arXiv:2308.08468 (2023).
- Anagnostopoulos et al., *Residual-based attention for PINNs*, arXiv:2307.00379 (2023).
- Vyas et al., *SOAP: Improving and stabilizing Shampoo*, arXiv:2409.11321 / 2502.00604.
- Repository: `wave/run_fno_baseline.py`, `src/operator_baselines.py`, `wave/fd_solver.py`, `wave/materials.py`, `wave/problem_data.py`, `final/ML/training_code_fno.py`; curated reading order in `papers/FNO_reading_order/`.